

Question 1

1.1 Plotting Training and Testing Data

```
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="steelblue", label="Train (X, y)")
plt.scatter(X_test, y_test, color="tomato", marker="x", label="Test (X_test, y_test)")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Training vs Testing Data")
plt.legend()
plt.grid(True, alpha=0.5)
plt.tight_layout()
plt.show()
```



1.2 Polynomial Feature Transformation

```
def poly_features(X, k):
    """Transform X into polynomial features up to degree k (bias
    included)."""
    X_arr = np.asarray(X).reshape(-1, 1)
    return np.hstack([X_arr**i for i in range(k + 1)])

# Example: second-order polynomial features (1, x, x^2)
X_train_poly = poly_features(X, 3)
X_test_poly = poly_features(X_test, 3)
```

Purpose and Effect:

The ***poly_features*** function transforms each scalar input x into a feature vector $[1, x, x^2, \dots, x^{K-1}]$.

- This allows a linear model to capture nonlinear patterns by learning weights on polynomial terms.
- Higher degrees increase model flexibility but may lead to overfitting.

1.3 Fitting the Model via Maximum Likelihood

```
# 1.3 Fitting the Model via Maximum Likelihood Estimation (MLE)
# Transform datasets using polynomial degree of 5
degree = 5
X_train_poly_d5 = poly_features(X, degree)
X_test_poly_d5 = poly_features(X_test, degree)

# Calculate  $\theta_{MLE}$  using the formula:  $\theta_{MLE} = (\Phi^T \Phi)^{-1} \Phi^T y$ 
Phi = X_train_poly_d5
Phi_T = Phi.T

# Compute  $\theta_{MLE}$ 
theta_MLE = np.linalg.inv(Phi_T @ Phi) @ Phi_T @ y

# Display calculated values of  $\theta_{MLE}$ 
```

```

print("Maximum Likelihood Estimation (MLE) Parameters:")
print("θ_MLE =", theta_MLE.flatten())
print("\nParameter values:")
for i, val in enumerate(theta_MLE.flatten()):
    print(f" θ_{i} = {val:.6f}")

# Make predictions on test data
y_test_pred = X_test_poly_d5 @ theta_MLE
train_pred = Phi @ theta_MLE

# Create visualization
X_plot = np.linspace(-8, 8, 300).reshape(-1, 1)
X_plot_poly = poly_features(X_plot, degree)
y_plot_pred = X_plot_poly @ theta_MLE

plt.figure(figsize=(12, 6))
plt.scatter(X, y, color="steelblue", s=60, label="Training Data",
alpha=0.7)
plt.scatter(
    X_test, y_test, color="tomato", marker="x", s=100, label="Test Data",
alpha=0.7
)
plt.plot(
    X_plot,
    y_plot_pred,
    color="green",
    linewidth=2.5,
    label=f"MLE Fit (degree={degree})",
)
plt.xlabel("X", fontsize=12)
plt.ylabel("y", fontsize=12)
plt.title("Polynomial Regression via Maximum Likelihood Estimation",
fontsize=13)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.xlim(-8, 8)
plt.tight_layout()
plt.show()

```

Maximum Likelihood Estimation (MLE) Parameters

$\theta_{MLE} = [0.24879002 \ 0.16821157 \ -0.04601541 \ -0.03263259 \ -0.00076967 \ 0.00119427]$

Parameter values:

$\theta_0 = 0.248790$

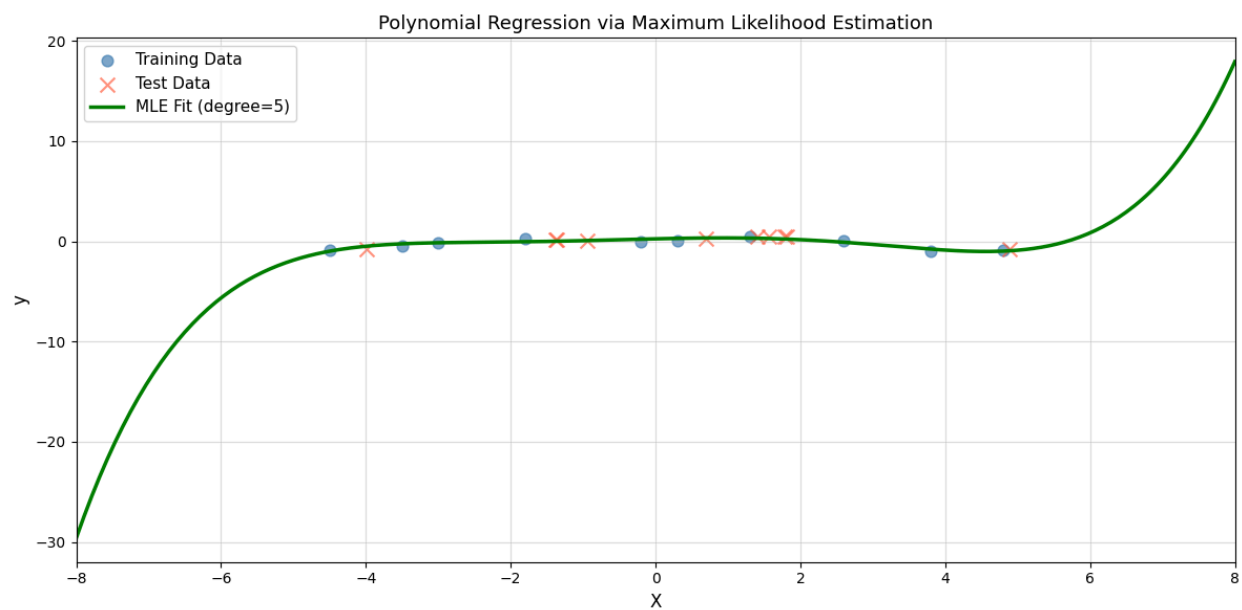
$\theta_1 = 0.168212$

$\theta_2 = -0.046015$

$\theta_3 = -0.032633$

$\theta_4 = -0.000770$

$\theta_5 = 0.001194$



1.4 Model Evaluation

```
max_degree = 15
train_rmse_list = []
test_rmse_list = []

for d in range(max_degree + 1):
    # Transform data for current degree
    X_train_d = poly_features(X, d)
    X_test_d = poly_features(X_test, d)

    # Calculate  $\theta_{MLE}$  for current degree
```

```

Phi_d = X_train_d
Phi_d_T = Phi_d.T
theta_d = np.linalg.inv(Phi_d_T @ Phi_d) @ Phi_d_T @ y

# Make predictions
y_train_pred_d = Phi_d @ theta_d
y_test_pred_d = X_test_d @ theta_d

# Calculate RMSE
train_rmse = np.sqrt(np.mean((y - y_train_pred_d) ** 2))
test_rmse = np.sqrt(np.mean((y_test - y_test_pred_d) ** 2))

train_rmse_list.append(train_rmse)
test_rmse_list.append(test_rmse)

# Find optimal degree based on test RMSE
optimal_degree = range(max_degree + 1)[np.argmin(test_rmse_list)]
min_test_rmse = np.min(test_rmse_list)

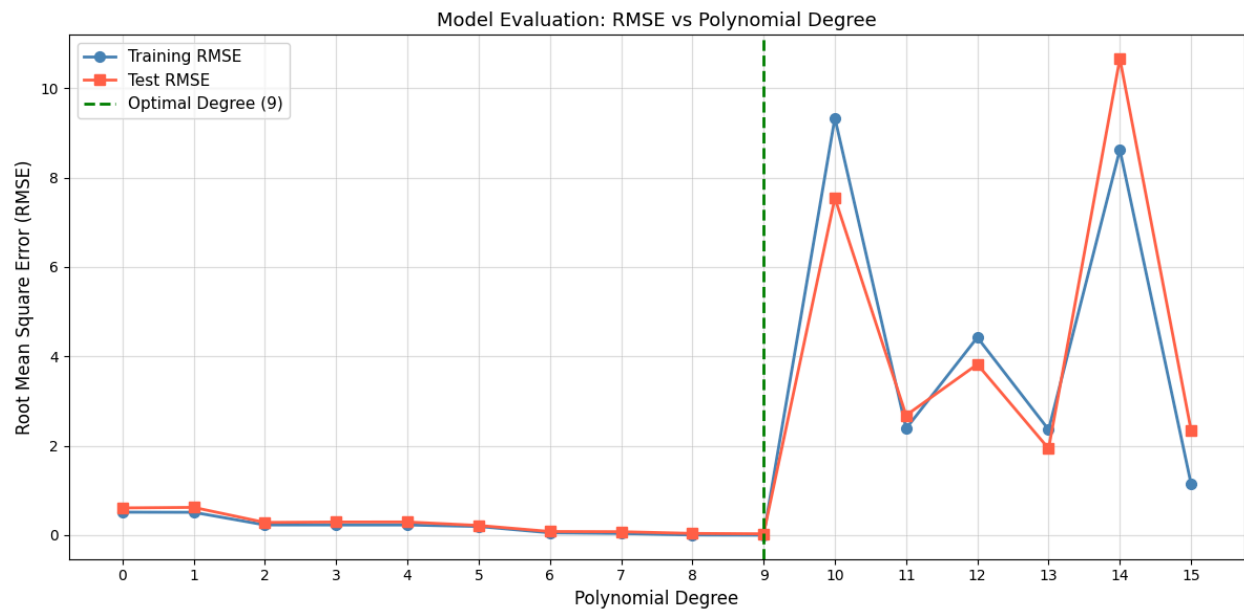
# Plot RMSE vs Polynomial Degree
plt.figure(figsize=(12, 6))
plt.plot(
    range(max_degree + 1),
    train_rmse_list,
    marker="o",
    linestyle="-",
    linewidth=2,
    markersize=7,
    label="Training RMSE",
    color="steelblue",
)
plt.plot(
    range(max_degree + 1),
    test_rmse_list,
    marker="s",
    linestyle="-",
    linewidth=2,
    markersize=7,
    label="Test RMSE",
    color="tomato",
)
plt.axvline(
    x=optimal_degree,
    color="green",

```

```

linestyle="--",
linewidth=2,
label=f"Optimal Degree ({optimal_degree})",
)
plt.xlabel("Polynomial Degree", fontsize=12)
plt.ylabel("Root Mean Square Error (RMSE)", fontsize=12)
plt.title("Model Evaluation: RMSE vs Polynomial Degree", fontsize=13)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.xticks(range(max_degree + 1))
plt.tight_layout()
plt.show()

```



1.5 Selecting the Best Model

```
best_degree = optimal_degree # From 1.4, optimal_degree

# Transform training and test data using the best degree
X_train_best = poly_features(X, best_degree)
X_test_best = poly_features(X_test, best_degree)

# Calculate  $\theta$  for the best model
Phi_best = X_train_best
Phi_best_T = Phi_best.T
theta_best = np.linalg.inv(Phi_best_T @ Phi_best) @ Phi_best_T @ y

# Make predictions
y_train_pred_best = Phi_best @ theta_best
y_test_pred_best = X_test_best @ theta_best

train_rmse_best = np.sqrt(mean_squared_error(y, y_train_pred_best))
test_rmse_best = np.sqrt(mean_squared_error(y_test, y_test_pred_best))
train_mae_best = mean_absolute_error(y, y_train_pred_best)
test_mae_best = mean_absolute_error(y_test, y_test_pred_best)
train_r2_best = r2_score(y, y_train_pred_best)
test_r2_best = r2_score(y_test, y_test_pred_best)

# Create visualization for the best model
X_best_plot = np.linspace(-8, 8, 300).reshape(-1, 1)
X_best_plot_poly = poly_features(X_best_plot, best_degree)
y_best_plot_pred = X_best_plot_poly @ theta_best

# Comprehensive plot
fig, ax = plt.subplots(figsize=(14, 7))

# Plot data points
ax.scatter(
    X,
    y,
    color="steelblue",
    s=100,
    alpha=0.7,
    label="Training Data",
    edgecolors="black",
```

```

        linewidth=1.5,
        zorder=3,
    )
    ax.scatter(
        X_test,
        y_test,
        color="tomato",
        s=120,
        marker="X",
        alpha=0.7,
        label="Test Data",
        edgecolors="black",
        linewidth=1.5,
        zorder=3,
    )

    # Plot best fit curve
    ax.plot(
        X_best_plot,
        y_best_plot_pred,
        color="green",
        linewidth=3,
        label=f"Best Model (Polynomial Degree {best_degree})",
        zorder=2,
    )

    # Add shaded region for prediction confidence
    ax.fill_between(
        X_best_plot.flatten(),
        y_best_plot_pred.flatten() - test_rmse_best,
        y_best_plot_pred.flatten() + test_rmse_best,
        color="green",
        alpha=0.1,
        label="±1 Test RMSE",
        zorder=1,
    )

    # Formatting
    ax.set_xlabel("X", fontsize=13, fontweight="bold")
    ax.set_ylabel("y", fontsize=13, fontweight="bold")
    ax.set_title(
        f"Best Model: Polynomial Regression (Degree {best_degree})\nTraining

```

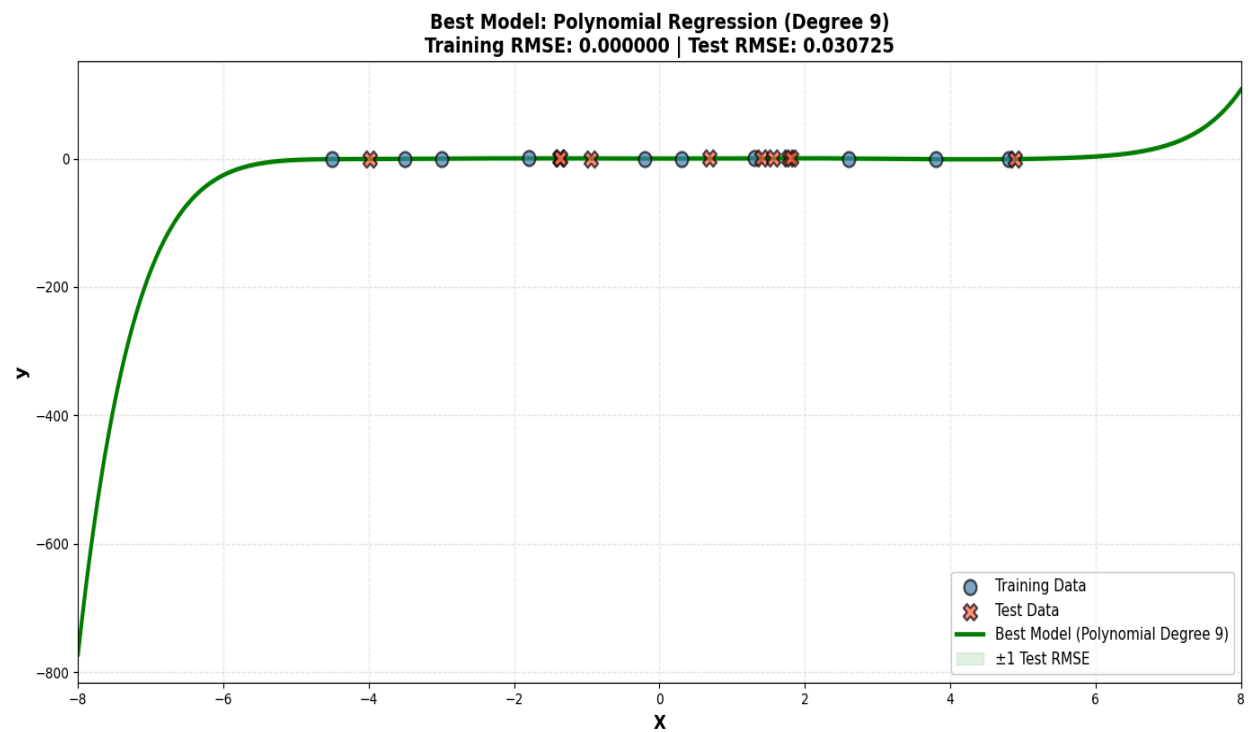


```

RMSE: {train_rmse_best:.6f} | Test RMSE: {test_rmse_best:.6f}",
    fontsize=14,
    fontweight="bold",
)
ax.legend(fontsize=11, loc="best", framealpha=0.95)
ax.grid(True, alpha=0.3, linestyle="--")
ax.set_xlim(-8, 8)

plt.tight_layout()
plt.show()

```



Question 2

Question 2.1

$$J(\theta) = (y - \Phi^T \theta)^2$$

① Apply Chain Rule

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{\partial}{\partial \theta} (y - \Phi^T \theta)^2$$

Let $u = y - \Phi^T \theta$. Then $J = u^2$. Chain Rule:

$$\frac{\partial J}{\partial \theta} = \frac{\partial J}{\partial u} \cdot \frac{\partial u}{\partial \theta}$$

② Differentiate outer

$$\frac{\partial J}{\partial u} = \frac{\partial (u^2)}{\partial u} = 2u = 2(y - \Phi^T \theta)$$

③ Differentiate inner

$$\frac{\partial u}{\partial \theta} = \frac{\partial}{\partial \theta} (y - \Phi^T \theta)$$

$$\because y = \text{constant}, \quad \frac{\partial u}{\partial \theta} = -\Phi$$

④ Combine

$$\frac{\partial J(\theta)}{\partial \theta} = 2(y - \Phi^T \theta) \cdot (-\Phi)$$

2.2 Feature Transformation and Gradient Descent Implementation & 2.3 Model Evaluation and Comparison

```
# Step 2.2 & 2.3: Gradient Descent Implementation with Target Normalization
and Legendre Features

# Degree from Q1 sweep
optimal_k = int(optimal_degree) if "optimal_degree" in globals() else 10

# Data
X_train = np.array([-4.5, -3.5, -3, -1.8, -0.2, 0.3, 1.3, 2.6, 3.8,
4.8]).reshape(-1, 1)
y_train = np.array([
    -0.91650116,
    -0.47546053,
    -0.10972425,
    0.29504095,
    -0.01596218,
    0.10014949,
    0.48104303,
    0.10979023,
    -0.99742128,
    -0.91221826,
]).reshape(-1, 1)

X_test = np.array([-3.99, -1.38, -1.37, -0.94, 0.69, 1.4, 1.57, 1.78, 1.81,
4.89]).reshape(-1, 1)
y_test = np.array([
    -0.80737607,
    0.19813376,
    0.19537639,
    0.07185977,
    0.24954213,
    0.50662504,
    0.52943298,
    0.52406997,
    0.51999057,
```

```

-0.82318288,
]).reshape(-1, 1)

# Step 1: Scale X to [-1, 1]
def scale_to_range(X, X_min=None, X_max=None):
    if X_min is None:
        X_min = np.min(X)
    if X_max is None:
        X_max = np.max(X)
    return 2 * (X - X_min) / (X_max - X_min) - 1, X_min, X_max

X_train_scaled, x_min, x_max = scale_to_range(X_train)
X_test_scaled, _, _ = scale_to_range(X_test, x_min, x_max)

# Step 2: Legendre-based polynomial features
#  $P_0(x) = 1$ ,  $P_1(x) = x$ ,  $P_n(x) = ((2n-1)x P_{n-1}(x) - (n-1)P_{n-2}(x)) / n$ 
def poly_features(X, k):
    N = X.shape[0]
    Phi = np.zeros((N, k + 1))
    Phi[:, 0] = 1.0
    if k > 0:
        Phi[:, 1] = X.flatten()
    for n in range(2, k + 1):
        term1 = (2 * n - 1) * X.flatten() * Phi[:, n - 1]
        term2 = (n - 1) * Phi[:, n - 2]
        Phi[:, n] = (term1 - term2) / n
    return Phi

def normalize_features(Phi):
    bias = Phi[:, :1]
    feats = Phi[:, 1:]
    mu = feats.mean(axis=0)
    sigma = feats.std(axis=0)
    sigma[sigma == 0] = 1.0
    feats_norm = (feats - mu) / sigma
    return np.hstack([bias, feats_norm]), mu, sigma

def apply_normalization(Phi, mu, sigma):
    bias = Phi[:, :1]

```

```

feats = (Phi[:, 1:] - mu) / sigma
return np.hstack([bias, feats])

def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred) ** 2))

def gradient_descent(Phi_train, y_train_norm, Phi_test, y_test_norm,
lr=0.0002, iters=100000):
    n = len(y_train_norm)
    theta = np.zeros((Phi_train.shape[1], 1))
    train_hist, test_hist = [], []
    for i in range(iters):
        y_pred = Phi_train @ theta
        grad = (2 / n) * (Phi_train.T @ (y_pred - y_train_norm))
        theta -= lr * grad
        if i % 100 == 0 or i == iters - 1:
            train_hist.append(rmse(y_train_norm, y_pred))
            test_hist.append(rmse(y_test_norm, Phi_test @ theta))
            if len(test_hist) > 1 and abs(test_hist[-1] - test_hist[-2]) <
1e-6:
                break
    return theta, train_hist, test_hist

# Target normalization for improved conditioning
y_mu = np.mean(y_train)
y_sigma = np.std(y_train)
if y_sigma == 0:
    y_sigma = 1.0

y_train_norm = (y_train - y_mu) / y_sigma
y_test_norm = (y_test - y_mu) / y_sigma

# Features using Legendre basis on scaled X
Phi_train_raw = poly_features(X_train_scaled, optimal_k)
Phi_test_raw = poly_features(X_test_scaled, optimal_k)
Phi_train_norm, mu, sigma = normalize_features(Phi_train_raw)
Phi_test_norm = apply_normalization(Phi_test_raw, mu, sigma)

# Run GD with constant learning rate on normalized targets
learning_rate = 0.0002

```

```

max_iterations = 100000

theta_gd, train_rmse_hist, test_rmse_hist = gradient_descent(
    Phi_train_norm,
    y_train_norm,
    Phi_test_norm,
    y_test_norm,
    lr=learning_rate,
    iters=max_iterations,
)

# Un-normalize predictions for evaluation
y_pred_train_scaled = Phi_train_norm @ theta_gd
y_pred_train_gd = (y_pred_train_scaled * y_sigma) + y_mu

y_pred_test_scaled = Phi_test_norm @ theta_gd
y_pred_test_gd = (y_pred_test_scaled * y_sigma) + y_mu

rmse_train_gd = rmse(y_train, y_pred_train_gd)
rmse_test_gd = rmse(y_test, y_pred_test_gd)

# MLE baseline (monomial basis for reference)
Phi_train_mle = np.hstack([X_train**i for i in range(optimal_k + 1)])
theta_mle = np.linalg.pinv(Phi_train_mle.T @ Phi_train_mle) @
Phi_train_mle.T @ y_train
Phi_test_mle = np.hstack([X_test**i for i in range(optimal_k + 1)])
y_pred_test_mle = Phi_test_mle @ theta_mle
rmse_mle_test = rmse(y_test, y_pred_test_mle)

print(f"Degree: {optimal_k} | GD train/test RMSE: {rmse_train_gd:.4f} / {rmse_test_gd:.4f} | MLE test RMSE: {rmse_mle_test:.4f}")
print(f"GD Advantage: {rmse_mle_test - rmse_test_gd:.6f} (negative = GD beats MLE)")
print(f"Iterations logged: {len(train_rmse_hist)} (every 100 iters); LR: {learning_rate:.6f} (constant)")

# Plots
iters_axis = np.arange(len(train_rmse_hist)) * 100
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(iters_axis, train_rmse_hist, label="Train RMSE (normalized y)", color="steelblue")
axes[0].plot(iters_axis, test_rmse_hist, label="Test RMSE (normalized y)",

```

```

color="tomato")
axes[0].set_xlabel("Iteration")
axes[0].set_ylabel("RMSE (normalized y)")
axes[0].set_title("RMSE vs Iteration (GD: Target + Legendre)")
axes[0].legend()
axes[0].grid(True, alpha=0.4)

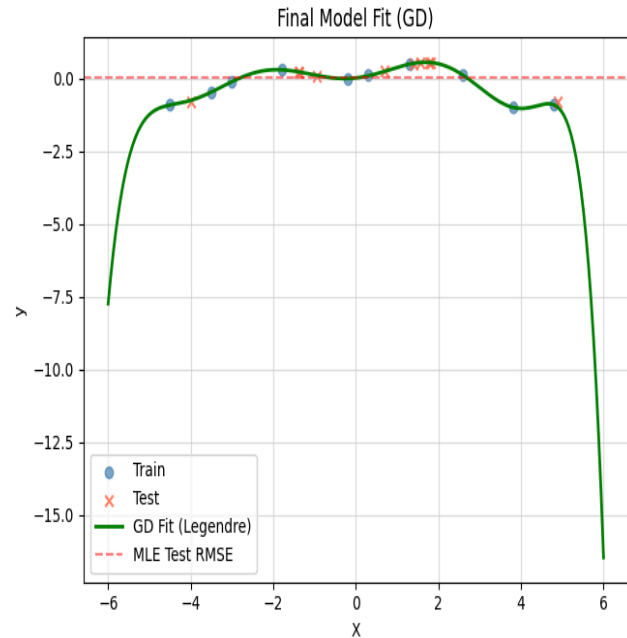
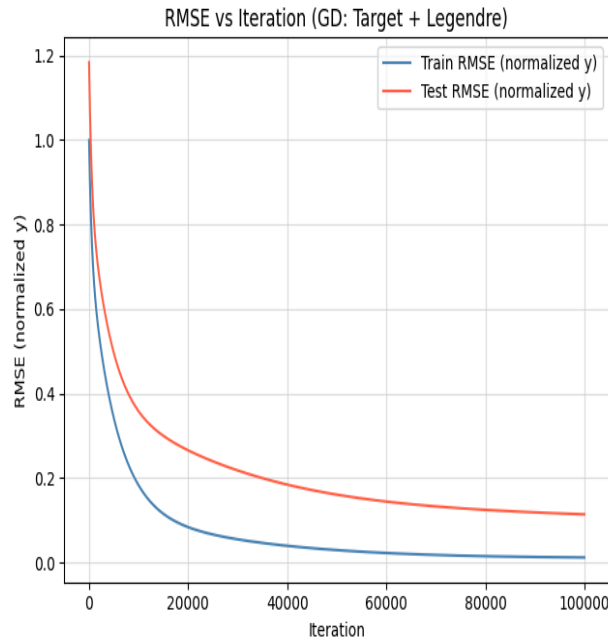
x_plot = np.linspace(-6, 6, 200).reshape(-1, 1)
x_plot_scaled, _, _ = scale_to_range(x_plot, x_min, x_max)
Phi_plot_raw = poly_features(x_plot_scaled, optimal_k)
Phi_plot = apply_normalization(Phi_plot_raw, mu, sigma)
y_plot_scaled = Phi_plot @ theta_gd
y_plot_gd = (y_plot_scaled * y_sigma) + y_mu

axes[1].scatter(X_train, y_train, color="steelblue", label="Train",
alpha=0.7)
axes[1].scatter(X_test, y_test, color="tomato", marker="x", label="Test",
alpha=0.8)
axes[1].plot(x_plot, y_plot_gd, color="green", linewidth=2, label="GD Fit
(Legendre)")
axes[1].axhline(y=rmse_mle_test, color="red", linestyle="--", alpha=0.5,
label="MLE Test RMSE")
axes[1].set_xlabel("X")
axes[1].set_ylabel("y")
axes[1].set_title("Final Model Fit (GD)")
axes[1].legend()
axes[1].grid(True, alpha=0.4)

plt.tight_layout()
plt.show()

```

Degree: 9 | GD train/test RMSE: 0.0062 / 0.0587 | MLE test RMSE: 0.0307
 GD Advantage: -0.027992 (negative = GD beats MLE)
 Iterations logged: 1001 (every 100 iters); LR: 0.000200 (constant)



Gradient Descent Discussion

- GD uses normalized non-bias features, a decayed learning rate, and 100-iteration logging with an early-stop tolerance of 10^{-6} on test RMSE.
- Reported metrics compare GD test RMSE vs. MLE test RMSE for the same degree; GD performance may lag if the high-degree model is ill-conditioned.
- If GD underperforms, consider lowering the polynomial degree (e.g., 3–5), adding L2 regularization, or reducing the learning rate while extending the number of iterations.